You said:



board.py
Code

game_engine.py
Code

main(20261015-06...
Code

I'm working on a Pygame match-3 gem game for a lab assignment. The game crashes on startup. I'll paste the error and the 3 files (main.py, game/board.py, game/game_engine.py). First, briefly summarize how the game flow works (swap, match detection, gravity/refill, scoring, moves). Then find the root cause of the crash, explain it in 2-3 lines, and give the exact fix. Also point out any other obvious bugs you notice, but don't fix anything beyond the crash yet. For every task in this chat: show only the changed functions with clear file names and where to put them, keep existing behavior and coding style intact, and don't refactor unrelated code.

Error:
NameError: name 'c' is not defined
File "game/board.py", line 119, in drop_and_refill
    if self.grid[r][c] is None:

ChatGPT said:

1. How your Match-3 game works

Based on your three uploaded files, the game flow is:

1. **Startup —** `main.py` : Initializes Pygame, creates the window and `GameEngine` , and starts the game loop.
2. **Board initialization —** `game/board.py` : Creates an 8×8 grid of randomly colored gems. It checks for existing matches and resolves them.
3. **Player input —** `game/game_engine.py` : The player clicks two adjacent gems to attempt a swap.
4. **Match detection —** `game/board.py` : `find_matches()` checks for three or more consecutive gems of the same color horizontally or vertically.
5. **Gravity and refill —** `game/board.py` : Matched gems are removed, gems above empty spaces fall downward, and new gems fill the empty cells.

6. **Scoring and moves:** Each cleared gem contributes 10 points. A valid or invalid adjacent swap consumes one move; a swap that produces no match is reversed.
7. **Win/loss and rendering:** The game wins at 500 points or loses when no moves remain. The engine updates animations and renders the board, score, moves, and result overlay.

2. Root cause of the startup crash

The error is in `game/board.py`, inside `drop_and_refill()`.

Your code uses `self.grid[r][c]`, but `c` **has never been defined in that function**. The method iterates over rows using `r`, but it does not iterate over columns. This causes the `NameError` during `Board.reset()` → `resolve_matches()` → `drop_and_refill()`, before the game window can finish initializing.

3. Exact fix — `game/board.py`

Replace the existing `drop_and_refill()` function, around lines 116–132, with the following function. **The fix is to add a loop over every column (`c`) and reset `empty_slots` separately for each column.**

```
def drop_and_refill(self):
    for c in range(GRID_SIZE):
        empty_slots = 0

        for r in range(GRID_SIZE - 1, -1, -1):
            if self.grid[r][c] is None:
                empty_slots += 1
            elif empty_slots > 0:
                gem = self.grid[r][c]
                gem.target_row = r + empty_slots
                gem.target_y = (r + empty_slots) * TILE_SIZE
                self.grid[r + empty_slots][c] = gem
                self.grid[r][c] = None

        for r in range(empty_slots):
            color = random.choice(GEM_COLORS)
            gem = Gem(color, r, c)
            gem.current_y = -((empty_slots - r) * TILE_SIZE)
            self.grid[r][c] = gem
```

Why this fixes it

- `for c in range(GRID_SIZE)` defines the column index before it is used.
- The existing bottom-to-top scan moves gems downward into empty cells.
- The refill loop creates new gems at the top of each column, preserving your existing falling animation.

You do **not** need to change `main.py` or `game/game_engine.py` to fix this startup crash.

4. Other obvious issues to keep in mind

I would leave these unchanged for now, as requested.

Location	Potential issue
`game/board.py` — `process_swap()`	An adjacent swap that produces no match still consumes a move, even though the swap is reversed. Check whether this matches your lab requirements.
`game/board.py` — `reset()`	Initial matches are cleared during board initialization, but those cleared gems do not add to the score.
`game/board.py` — `find_matches()`	The implementation checks horizontal and vertical matches, but does not detect diagonal matches. This is normal for many match-3 games, so change it only if your lab requires diagonals.

5. Test the fix

1. Save `game/board.py`.
2. Run the same command from your project directory:

   ```
   py main.py
   ```

3. Check that the board initializes and displays without the `NameError`.
4. Try swapping adjacent gems to create a match and verify that the gems fall, new gems appear, the score increases, and the move count updates.

For now, change only `drop_and_refill()`. If another error appears after this fix, share the traceback and we can address that separately without refactoring the rest of your project.

You said:



Can you help me debug this?

Worked for 11s

ChatGPT said:

Yes! This is a **different error from the previous one**, and it points to a likely indentation or method-placement problem in `game/board.py`.

1. Root cause

Your `Board.__init__()` calls `self.reset()`, and `reset()` calls `self.resolve_matches()`.

The error `AttributeError: 'Board' object has no attribute 'resolve_matches'` means Python cannot find `resolve_matches()` as a method of the `Board` class. It may have become accidentally nested inside another function when you edited `drop_and_refill()`.

2. Exact fix — `game/board.py`

Find `drop_and_refill()` and make sure the following two functions are placed **one after the other inside** `class Board`, with four spaces before each `def`. Replace the existing versions of both functions with these:

```
    def drop_and_refill(self):
        for c in range(GRID_SIZE):
            empty_slots = 0

            for r in range(GRID_SIZE - 1, -1, -1):
                if self.grid[r][c] is None:
                    empty_slots += 1
                elif empty_slots > 0:
                    gem = self.grid[r][c]
                    gem.target_row = r + empty_slots
                    gem.target_y = (r + empty_slots) * TILE_SIZE
                    self.grid[r + empty_slots][c] = gem
                    self.grid[r][c] = None

            for r in range(empty_slots):
                color = random.choice(GEM_COLORS)
                gem = Gem(color, r, c)
                gem.current_y = -((empty_slots - r) * TILE_SIZE)
                self.grid[r][c] = gem
```

```
    def resolve_matches(self):
        total_cleared = 0
        while True:
            matches = self.find_matches()
            if not matches:
                break
            total_cleared += len(matches)
            for r, c in matches:
                self.grid[r][c] = None
            self.drop_and_refill()
        return total_cleared
```

Important: Both `def` lines must have exactly the same indentation as `def reset()`, `def find_matches()`, and `def process_swap()`. The contents of each function must be indented one level further.

3. Test again

Save `game/board.py`, then run:

```
py main.py
```

If the error persists, send me a screenshot or paste the code from `def drop_and_refill(self):` through `def process_swap(self, pos1, pos2):`. That will let me identify the exact indentation issue without changing unrelated code.

You said:

board(1).py
Code

game_engine(1).py
Code

main(20261015-07...
Code

Task 1: Fix the bug. When I swap two gems that do not form any 3-in-a-row, the gems revert to their original positions but the game still deducts a move. Moves should only be deducted when the swap creates at least one valid match. Find the root cause in the code, explain it in 2-3 lines, and give the exact fix. Also confirm that valid swaps still deduct exactly 1 move, and that the win (500 points) and OUT OF MOVES states still work.

ChatGPT said:

1. Root cause — `game/board.py`

In your current `process_swap()` function, `self.moves_remaining -= 1` executes **before** the code checks whether the swap creates a match. Consequently, even when `find_matches()` returns no matches and the gems are swapped back, a move has already been deducted.

The fix is to deduct the move **only after confirming that at least one match exists**. Your existing scoring and game-result checks can remain unchanged.

2. Exact fix — replace `process_swap()` in `game/board.py`

Replace the existing function around lines 150–165 with this version:

```
    def process_swap(self, pos1, pos2):
        if not self.is_adjacent(pos1, pos2) or self.is_game_over() or
self.is_animating():
            return False

        self.swap_gems(pos1, pos2)
        matches = self.find_matches()

        if not matches:
            self.swap_gems(pos1, pos2)
            return False

        self.moves_remaining -= 1

        cleared = self.resolve_matches()
        self.score += cleared * 10
        return True
```

3. Expected behavior after the fix

Scenario	Expected result
Adjacent swap creates no match	Gems revert; **0 moves deducted**
Adjacent swap creates a match	Match resolves; **exactly 1 move deducted**
Score reaches 500	`check_result()` returns `"WIN"` and displays `STAGE CLEARED!`

Moves reach 0 before winning	`check_result()` returns `"LOSS"` and displays `OUT OF MOVES!`
Non-adjacent swap	Rejected; 0 moves deducted

Win/loss confirmation: The existing `is_game_over()` and `check_result()` functions already contain the required 500-point and zero-move conditions. This fix does not modify them, so those rules remain intact by code inspection.

I haven't executed the game, so test one valid swap and one invalid swap to verify the behavior in your environment. No changes to `main.py` or `game/game_engine.py` are needed for this task.

You said:

Task 2: Implement a cascade combo multiplier. Currently every cleared gem gives flat points. Add a multiplier that increases for each consecutive cascade (chain reaction from gems falling and forming new matches) within a single player turn: the first match is x1, the first cascade x2, the next x3, and so on. Reset the multiplier to 1 at the start of every new turn. Show the current combo (e.g. "COMBO x3") on screen briefly when a cascade happens. Don't break Task 1's fix. Give only the changed code.

ChatGPT said:

`game/board.py` — **replace** `__init__()`

Keep your existing initialization and add the combo attributes shown here.

```
    def __init__(self, offset_x, offset_y, target_score=500, max_moves=20):
        self.offset_x = offset_x
        self.offset_y = offset_y
        self.target_score = target_score
        self.max_moves = max_moves
        self.grid = [[None for _ in range(GRID_SIZE)] for _ in range(GRID_SIZE)]
        self.selected = None
        self.score = 0
        self.moves_remaining = max_moves

        self.combo_multiplier = 1
        self.combo_display = None
        self.combo_display_until = 0
```

```
        self.reset()
```

game/board.py — **replace** `reset()`

```
    def reset(self):
        self.score = 0
        self.moves_remaining = self.max_moves
        self.selected = None
        self.combo_multiplier = 1
        self.combo_display = None
        self.combo_display_until = 0

        for r in range(GRID_SIZE):
            for c in range(GRID_SIZE):
                color = random.choice(GEM_COLORS)
                gem = Gem(color, r, c)
                gem.current_y = gem.target_y
                self.grid[r][c] = gem

        self.resolve_matches()
```

game/board.py — **replace** `resolve_matches()`

The initial match scores x1, the next cascade x2, and each subsequent cascade increases the multiplier by one. Initial board cleanup still awards no points.

```
    def resolve_matches(self, score_matches=False):
        total_cleared = 0
        multiplier = 1

        while True:
            matches = self.find_matches()
            if not matches:
                break

            self.combo_multiplier = multiplier

            if score_matches:
                self.score += len(matches) * 10 * multiplier
```

```
                if multiplier >= 2:
                    self.combo_display = multiplier
                    self.combo_display_until = pygame.time.get_ticks() + 1500

            total_cleared += len(matches)

            for r, c in matches:
                self.grid[r][c] = None

            self.drop_and_refill()
            multiplier += 1

        return total_cleared
```

`game/board.py` — **replace** `process_swap()`

This preserves Task 1: invalid swaps are reversed without consuming a move or awarding points.

```
    def process_swap(self, pos1, pos2):
        if not self.is_adjacent(pos1, pos2) or self.is_game_over() or
self.is_animating():
            return False

        self.combo_multiplier = 1
        self.combo_display = None
        self.combo_display_until = 0

        self.swap_gems(pos1, pos2)
        matches = self.find_matches()

        if not matches:
            self.swap_gems(pos1, pos2)
            return False

        self.moves_remaining -= 1

        self.resolve_matches(score_matches=True)
        return True
```

`game/game_engine.py` — **add inside** `render()`

Insert this block immediately after `self.board.render(screen)` :

```
        if (
            self.board.combo_display is not None
            and pygame.time.get_ticks() < self.board.combo_display_until
        ):
            combo_surf = self.font_big.render(
                f"COMBO x{self.board.combo_display}", True, (255, 220, 80)
            )
            screen.blit(
                combo_surf,
                (
                    self.width // 2 - combo_surf.get_width() // 2,
                    self.board.offset_y - 30,
                ),
            )
```

You said:

Task 3: Implement 4-in-a-row special line-clear gems. When a swap creates a match of exactly 4 matching gems in a line, instead of clearing all four, turn one of them (the swapped gem's position) into a special glowing gem of the same color, drawn with a visible glow or border effect. When this special gem is later part of any match, it detonates and clears its entire row (if the 4-match was horizontal) or entire column (if vertical), and those cleared gems score points too. Make sure gravity/refill and cascades still work correctly, and that the combo multiplier from Task 2 still works. Give only the changed code.

Worked for 20s

You said:

Task 4: Implement an idle hint indicator. If the player has not clicked anything for more than 5 seconds, find a valid adjacent swap that would create a match and highlight those two gems with a subtle pulsing or shimmering animation. Remove the hint as soon as the player clicks, and restart the 5-second idle timer after every move. Don't show hints while gems are animating or when the game is

over (win/loss overlay). If no valid swap exists, do nothing. Give only the changed code.

ChatGPT said:

`game/board.py` — **add inside** `Board`

This helper finds a valid adjacent swap by temporarily swapping each pair, checking for a match, and restoring the original board. It does not consume moves or change the score.

```
    def find_valid_swap(self):
        for r in range(GRID_SIZE):
            for c in range(GRID_SIZE):
                for dr, dc in ((0, 1), (1, 0)):
                    nr, nc = r + dr, c + dc

                    if nr >= GRID_SIZE or nc >= GRID_SIZE:
                        continue

                    pos1 = (r, c)
                    pos2 = (nr, nc)

                    self.swap_gems(pos1, pos2)
                    has_match = bool(self.find_matches())
                    self.swap_gems(pos1, pos2)

                    if has_match:
                        return pos1, pos2

        return None
```

`game/game_engine.py` — **add to** `__init__()`

Insert after `self.board = Board(...)`.

```
        self.idle_since = pygame.time.get_ticks()
        self.hint_swap = None
```

game/game_engine.py — **replace** handle_click()

```
    def handle_click(self, mouse_pos):
        self.idle_since = pygame.time.get_ticks()
        self.hint_swap = None
        self.board.hint_swap = None

        if self.board.is_game_over() or self.board.is_animating():
            return

        mx, my = mouse_pos
        bx = mx - self.board.offset_x
        by = my - self.board.offset_y

        if 0 <= bx < GRID_SIZE * TILE_SIZE and 0 <= by < GRID_SIZE * TILE_SIZE:
            col = int(bx // TILE_SIZE)
            row = int(by // TILE_SIZE)

            if self.board.selected is None:
                self.board.selected = (row, col)
            else:
                prev_selected = self.board.selected
                if prev_selected == (row, col):
                    self.board.selected = None
                else:
                    self.board.process_swap(prev_selected, (row, col))
                    self.board.selected = None
```

game/game_engine.py — **replace** update()

```
    def update(self):
        self.board.update()

        if self.board.is_game_over() or self.board.is_animating():
            self.hint_swap = None
            self.board.hint_swap = None
            return

        if pygame.time.get_ticks() - self.idle_since > 5000:
            if self.hint_swap is None:
```

```
                self.hint_swap = self.board.find_valid_swap()
                self.board.hint_swap = self.hint_swap
```

`game/game_engine.py` — **replace** `reset()`

```
    def reset(self):
        self.board.reset()
        self.idle_since = pygame.time.get_ticks()
        self.hint_swap = None
        self.board.hint_swap = None
```

`game/board.py` — **add to** `__init__()`

Insert this line after the existing board state attributes:

```
        self.hint_swap = None
```

`game/board.py` — **add at the end of** `render()`

Place this after the existing grid-rendering loops, at the same indentation level as those loops.

```
        if self.hint_swap and not self.is_animating():
            pulse = (pygame.time.get_ticks() % 1000) / 1000
            glow_width = 2 if pulse < 0.5 else 3

            for r, c in self.hint_swap:
                x = self.offset_x + c * TILE_SIZE
                y = self.offset_y + r * TILE_SIZE
                hint_rect = pygame.Rect(
                    x + 4, y + 4, TILE_SIZE - 8, TILE_SIZE - 8
                )
                pygame.draw.rect(
                    surface,
                    (180, 255, 200),
                    hint_rect,
                    width=glow_width,
                    border_radius=10,
                )
```

This keeps the hint hidden during animations and game-over states, clears it as soon as the player clicks, and searches for a possible move only after five seconds of inactivity. If no valid swap exists, no hint is drawn. No changes to `main.py` are required.